

gimbalAnglesToMotorAngles

Executive Summary

Given an incoming set of gimbal angles (ψ, ϕ) , this module computes the corresponding stepper motor angles (θ_1, θ_2) which map to the requested gimbal angles. The motor angles are determined through interpolation, using provided gimbal-to-motor lookup data tables. The algorithm flow is outlined in Figure 1 below.

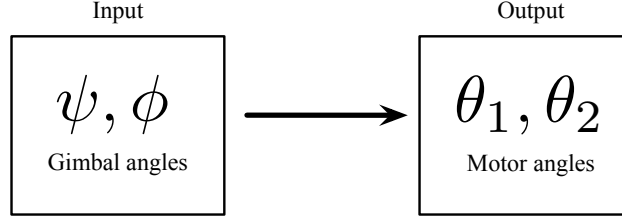


Figure 1: Illustration of the algorithm flow.

Module Architecture

The module is split into two layers:

- The **adapter** (`gimbalAnglesToMotorAngles.h/.cpp`) is the SysModel-derived class that handles message I/O, validates configuration, builds an immutable `GimbalAnglesToMotorAnglesConfig` from public properties, and constructs the algorithm via two-phase initialization.
- The **algorithm** (`gimbalAnglesToMotorAnglesAlgorithm.h/.cpp`) is a pure C++23 class with no framework dependencies. It takes the requested gimbal angles as input, computes the corresponding stepper motor angles, and returns a payload struct as output. It must not throw from `update()`.

A pure-C shim (`gimbalAnglesToMotorAnglesAlgorithm_c.h/.cpp`) wraps the algorithm class for use by Ada/Adamant components via `extern "C"` bindings.

Adapter Layer

The adapter consumes the following messages and public configuration properties:

Table 1: Module I/O Messages

Msg Variable Name	Msg Type	Description
<code>twoAxisGimbalOutMsg</code>	<code>TwoAxisGimbalMsgF32Payload</code>	Input for the two gimbal angles
<code>motor1AngleOutMsg</code>	<code>HingedRigidBodyMsgF32Payload</code>	Output for motor 1 angles
<code>motor2AngleOutMsg</code>	<code>HingedRigidBodyMsgF32Payload</code>	Output for motor 2 angles

Table 2: Module Configuration Properties

Parameter Name	Type	Units	Default	Description	Bounds
minAngle	float	[rad]	0	Lower bound of the motor's travel range; reference angles below this are rejected	Must be in $[-2\pi, 2\pi]$ and strictly less than maxAngle (validated at reset())
maxAngle	float	[rad]	2π	Upper bound of the motor's travel range; reference angles above this are rejected	Must be in $[-2\pi, 2\pi]$ and strictly greater than minAngle (validated at reset())
gimbalToMotor1AngleData	std::array<float, 4086>	[rad]	[-]	Gimbal-to-motor 1 angle interpolation table	Values must be within the provided motor angle range (minAngle, maxAngle). Gimbal angles cannot exceed ± 90 degrees
gimbalToMotor2AngleData	std::array<float, 4086>	[rad]	[-]	Gimbal-to-motor 2 angle interpolation table	Values must be within the provided motor angle range (minAngle, maxAngle). Gimbal angles cannot exceed ± 90 degrees
rowStartStrideIndices	std::array<int, 109>	[-]	[-]	Stride indices for the starting location of the table rows	Values cannot be negative and entries must be ascending
rowStartColIndices	std::array<int, 109>	[-]	[-]	Column indices for the starting location of the table rows	Values cannot be negative or exceed the number of table columns
tipColIdxOffset	int	[-]	[-]	Table index corresponding to zero gimbal angle 1	Cannot exceed number of table columns
tiltRowIdxOffset	int	[-]	[-]	Table index corresponding to zero gimbal angle 2	Cannot exceed number of table rows

... continued on next page

Table 2: Module Configuration Properties (... continued)

Parameter Name	Type	Units	Default	Description	Bounds
tableStepAngle	float	[rad]	[-]	Interpolation table motor discretization step angle	Cannot be negative or zero

Algorithm Layer

Mathematical Formulation

The stepper motor angles corresponding to the requested gimbal angles are determined using two gimbal-to-motor angle lookup tables. One lookup table is provided for each stepper motor. The structure of these tables is outlined in Figure 2.

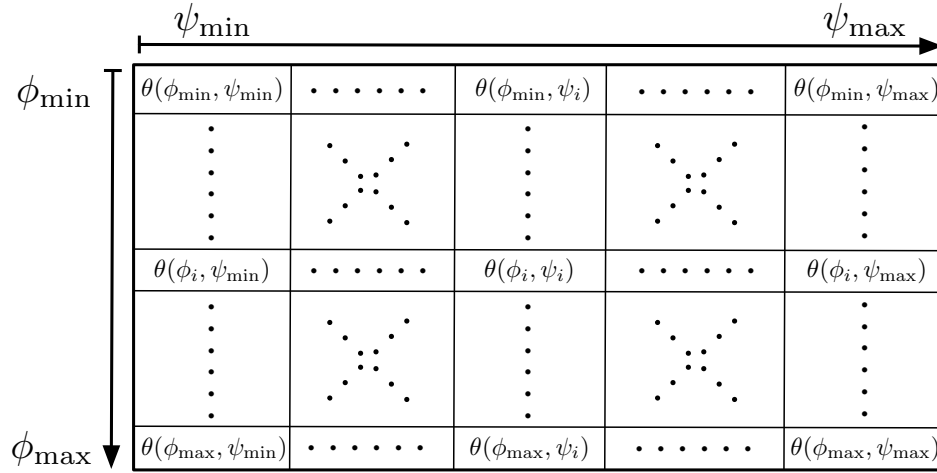


Figure 2: Gimbal-to-motor angle lookup table data format.

Implicitly, each table varies the gimbal tip and tilt angles in ascending order along the columns and rows, respectively. Both tables discretize the gimbal angles by the fixed configuration input value given. The motor angles corresponding to the discrete combination of gimbal angles are explicitly provided in each table. Accordingly, the motor angles corresponding to the minimum gimbal angles are located in the upper-left entry of each table and the angles corresponding to the maximum gimbal angles are located in the bottom-right entry of each table. The bilinear interpolation helper function is used to obtain the motor angles.

Determine Bounding Angles for Interpolation

First, the upper and lower bounds for both gimbal angles are determined as the nearest

multiple of the fixed table step angle δ .

$$\begin{aligned} \psi_L \leq \psi \leq \psi_U \quad \text{where} \quad \psi_L &= \delta \left\lfloor \frac{\psi}{\delta} \right\rfloor, \quad \psi_U = \delta \left\lceil \frac{\psi}{\delta} \right\rceil \\ \phi_L \leq \phi \leq \phi_U \quad \text{where} \quad \phi_L &= \delta \left\lfloor \frac{\phi}{\delta} \right\rfloor, \quad \phi_U = \delta \left\lceil \frac{\phi}{\delta} \right\rceil \end{aligned}$$

Pull Motor Angles From Tables

Using the bounding gimbal angles, the corresponding bounding motor angles are extracted from the provided tables using the `pullAngles` method.

$$\begin{aligned} (\theta_{1,LL}, \theta_{2,LL}) &= \text{pullAngles}(\psi_L, \phi_L) \\ (\theta_{1,LU}, \theta_{2,LU}) &= \text{pullAngles}(\psi_L, \phi_U) \\ (\theta_{1,UL}, \theta_{2,UL}) &= \text{pullAngles}(\psi_U, \phi_L) \\ (\theta_{1,UU}, \theta_{2,UU}) &= \text{pullAngles}(\psi_U, \phi_U) \end{aligned}$$

The `pullAngles` method first determines the integer table indices required to pull the correct motor angles from the lookup tables. The required table row and column indices i and j are determined as:

$$(i, j) = \left(\frac{\phi + \text{tiltRowIdxOffset}\delta}{\delta}, \frac{\psi + \text{tipColIdxOffset}\delta}{\delta} \right)$$

Note from the equation above that the incoming gimbal angles must be shifted to determine the correct row and column lookup table indices. The motor angles corresponding to zero gimbal angles are located at the center of the lookup tables (For $(\psi, \phi) = (0, 0)$, the required table row and column indices are $(i, j) = (\text{tiltRowIdxOffset}, \text{tipColIdxOffset})$).

Perform Interpolation

Using (1) the required gimbal angles (ψ, ϕ) , (2) bounding gimbal angles $(\psi_L, \psi_U, \phi_L, \phi_U)$, and (3) the bounding motor angles, interpolation is performed using the bilinear interpolation helper method. Note that additional logic is implemented in the helper method if either linear interpolation or no interpolation are instead required. The bilinear and linear interpolation formula are shown below.

- Bilinear interpolation:

$$\begin{aligned} \theta = \frac{1}{(\psi_U - \psi_L)(\phi_U - \phi_L)} & \left(\theta_{LL}(\psi_U - \psi)(\phi_U - \phi) + \theta_{UL}(\psi - \psi_L)(\phi_U - \phi) \right. \\ & \left. + \theta_{LU}(\psi_U - \psi)(\phi - \phi_L) + \theta_{UU}(\psi - \psi_L)(\phi - \phi_L) \right) \end{aligned}$$

- Linear interpolation:

$$\theta = \frac{\theta_L(\beta_U - \beta)}{\beta_U - \beta_L} + \frac{\theta_U(\beta - \beta_L)}{\beta_U - \beta_L}$$

where $\beta = \psi$ or ϕ depending on the bounded gimbal angle.

Algorithm Assumptions and Limitations

- All bounding gimbal angles are required to fall within the lookup table bounds. Requests which fall along an edge of the interpolation table are invalid.
- For invalid gimbal angle requests, the previously output valid motor angles are returned as the algorithm default output. On the first pass, an invalid request returns the motor angles corresponding to the gimbal home (0,0) position.

Test Description

The module is verified through regression tests that compare the algorithm results against an independent reference implementation. Setup tests are used for the `GimbalAnglesToMotorAnglesConfig` validators and round-trip tests are used to check the set configuration variables are correctly returned. Fuzz tests are added for the regression and property tests, where the configuration and inputs are randomized over reasonable ranges. The fuzz tests generate independent lookup tables and associated table layout data. A second test file `test_gimbalAnglesToMotorAnglesImportTables` is added which imports the lookup table data and associated table layout data from CSV files. Hand-pulled data from the CSV files is used to perform the reference interpolation and the results are compared with the algorithm results.